Problem 14: Real-Time Edge Vision System for Traffic Monitoring and Incident Detection
Domain: Computer Vision | Edge AI | Model Compression | Smart Cities
Difficulty: Medium-Hard to Hard
Status: NEW (added to reach 15)

Problem

Cities need real-time traffic analytics (counts, speeds, congestion, incidents) but cannot stream all video to the cloud due to bandwidth, latency and privacy. Models that work on a GPU server often fail on low-power devices and under night, rain or camera-angle changes.

Objective

Build an edge-deployable pipeline that detects and tracks vehicles, estimates traffic metrics, detects incidents, and demonstrates a measured accuracy-latency trade-off after model compression.

Users

Traffic authorities, smart-city operators.

Core Requirements

- Ingest video streams or recorded clips.
- Vehicle detection (cars, bikes, buses, trucks, pedestrians).
- Multi-object tracking with persistent IDs.
- Counting by lane/zone and speed estimate (with camera calibration or homography).
- Congestion level classification.
- Incident detection (stopped vehicle, wrong-way driving, sudden slowdown).
- Model compression: quantization and/or pruning/distillation; export to ONNX/TensorRT/TFLite.
- Run on an edge target (Jetson, Raspberry Pi + accelerator, or CPU-limited simulation) and report FPS and latency.
- Privacy: blur faces/plates before any storage or transmission.
- Dashboard with live counts, alerts and clips.

Advanced Challenges

- Robustness to night, rain, glare, occlusion; domain adaptation to Indian traffic (dense, mixed vehicles).
- Accuracy-latency-power Pareto analysis across model sizes.
- Handling ID switches in crowded scenes.
- Event detection with temporal models rather than hand-coded rules only.
- Sending only metadata/events to the cloud (edge-cloud split).
- Active learning loop: send hard frames for annotation.

Suggested Datasets

BDD100K, UA-DETRAC, VisDrone, MOT17/MOT20 (tracking), COCO (pretraining), India Driving Dataset (IDD).

Suggested Tech

Python, PyTorch, Ultralytics YOLO (or RT-DETR), ByteTrack/BoT-SORT, OpenCV, ONNX Runtime, TensorRT/TFLite, FastAPI, MQTT, Streamlit.

Reference Architecture

Camera/video → decoder → detector (compressed) → tracker → analytics (count, speed, congestion, incident rules/model) → privacy filter → event publisher (MQTT) → cloud dashboard.

Evaluation

mAP@0.5 and mAP@0.5:0.95, MOTA/IDF1/HOTA for tracking, counting error vs manual count, incident precision/recall, FPS, latency per frame, model size, accuracy drop after

quantization, performance by weather/lighting slice.

Safety Note

Respect privacy and local law; do not store identifiable faces or plates without authorization.

Pitfalls

Reporting FPS on a desktop GPU only; training/testing on the same camera view; speed estimates without calibration.

Suggested Timeline (48h)

0-8h baseline detector + tracker; 8-20h analytics + incidents; 20-30h compression + edge benchmark; 30-40h robustness slices; 40-48h dashboard/demo.

Standard Judging Rubric (applies to every problem)

Problem understanding	10%
Technical depth	20%
Data engineering (quality, leakage prevention, reproducibility)	10%
Model performance (metrics, baselines, honest evaluation)	15%
System architecture	15%
Working prototype	10%
Explainability and responsible AI	10%
Innovation and scalability	5%
Presentation and demo	5%

Mandatory Submission Checklist

- □ GitHub repository with setup instructions
- □ Working prototype or recorded demo
- □ Documented system architecture
- □ Dataset sources and preprocessing methodology
- □ Evaluation results with baseline comparisons
- □ Limitations and failure cases
- □ One technically challenging feature demonstrated
- □ Short presentation: problem, approach, results, future work